

PROJECT REPORT

Supply Chain & Sales Intelligence System

An Automated Data Engineering Pipeline and Interactive Business Intelligence Workspace for Unifying Sales & Inventory Data, Modelling Asset Optimization, and Mitigating Fulfillment Risk

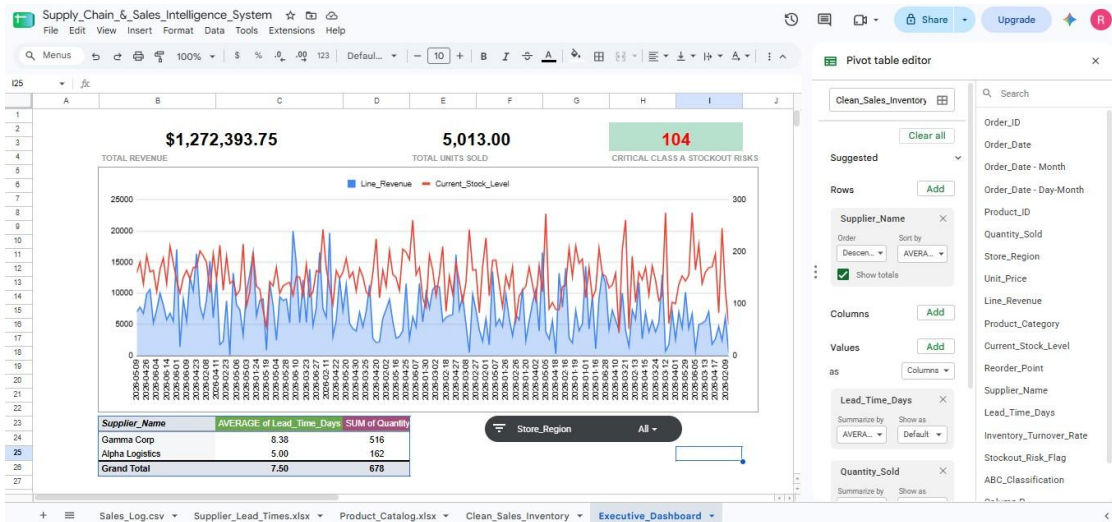


Figure 0 — The Completed Supply Chain & Sales Intelligence Workspace (Executive Dashboard View)

Prepared by: Robinson

Focus Area: Data Engineering, Business Intelligence & Data-Driven Risk Analysis

Tools Used: Python (Pandas, NumPy), Google Colab, Google Sheets

Table of Contents

1. Executive Summary.....	3
2. Business Case & Problem Statement.....	3
2.1 Pre-Transformation Vulnerability Analysis.....	4
3. Solution Architecture & Technical Stack.....	6
3.1 Building Realistic Test Data (Mock Data Simulation).....	6
4. The Algorithmic Data Engineering Pipeline.....	8
4.1 Step-by-Step Breakdown of the Cleaning Process.....	8
4.2 Business Metrics Calculated Automatically.....	9
5. Dashboard Architecture & Layout Blueprint.....	11
5.1 Row 1 — Multi-Tier Strategic KPI Banner.....	11
5.2 Row 2 — Tactical Supply vs. Demand Velocity Trends.....	11
5.3 Row 3 — Actionable Operational Intelligence Tables.....	12
5.4 Interactive Filtering — The Store Region Slicer.....	12
6. How Everything Connects — A Simple Walkthrough.....	13
7. Conclusion.....	14
8. Strategic Recommendations.....	14
Appendix A: Technical Stack Summary.....	14
Appendix B: Key Output Files.....	15
Appendix C: Full Step-by-Step Replication Manual.....	16

1. Executive Summary

This report documents the design, build, and outcome of a complete, end-to-end data project: the Supply Chain & Sales Intelligence System. In simple terms, this project takes messy, scattered business records — sales receipts, warehouse stock counts, and supplier delivery schedules kept in separate files — and turns them into a single, clean, trustworthy source of truth. That clean data is then used to calculate key business numbers automatically, and finally displayed on an easy-to-read, interactive dashboard that a manager can use to make fast, confident decisions.

The project was carried out in four broad stages: (1) simulating and inspecting realistic but messy source data, (2) building an automated Python pipeline to clean, merge, and calculate metrics from that data, (3) designing an executive dashboard in Google Sheets to visualise the results, and (4) translating the findings into concrete business recommendations.

The finished system revealed that, out of a full inventory catalogue, 104 individual products classified as top business priority ("Class A") were at immediate risk of running out of stock, and that one supplier — Gamma Corp — was responsible for the majority of the shipping delays behind that risk, with deliveries taking an average of 8.38 days against a faster 5-day average from the alternative supplier, Alpha Logistics.

1.1 Why This Project Matters (In Plain Terms)

Imagine a shop that sells thousands of different products, supplied by several different vendors, across several regional store locations. If the shop's sales records, warehouse stock levels, and supplier delivery times are all kept in separate spreadsheets that nobody ever combines, then nobody can see the full picture. A product might be selling extremely fast in one region while quietly running out of stock, and nobody notices until a customer complains that the item is unavailable. This project solves exactly that problem: it automatically pulls all three data sources together, checks the health of every product, and raises a flag the moment something needs attention.

2. Business Case & Problem Statement

Modern businesses generate enormous volumes of day-to-day transactional data — every sale, every stock movement, every delivery. On their own, none of these records tell the full story. The core problem this project addresses is called "data fragmentation": when related pieces of information live in separate, disconnected files, decision-makers cannot see how they relate to one another, and important warning signs get missed.

Without a centralised system pulling everything together, two costly problems tend to hide in plain sight:

- **Stockouts on fast-selling items:** Popular products run out of stock without anyone noticing early enough to reorder in time, leading to lost sales (this is called "revenue leakage").
- **Excess slow-moving inventory:** Products that barely sell keep getting ordered and stored anyway, tying up money in stock that just sits on a shelf instead of being available for more useful business spending (this is called "trapped working capital").

This project's purpose was to replace slow, manual, error-prone spreadsheet auditing with an automated engine that cleans the data, calculates the metrics that matter, and displays them on a live dashboard — so that these two problems can be caught and acted on early, instead of being discovered after the damage is done.

2.1 Pre-Transformation Vulnerability Analysis

Before any cleaning or calculation could happen, the raw, unrefined data first had to be inspected closely to understand exactly what was wrong with it. This step is a bit like a doctor examining a patient before deciding on treatment — you cannot fix a problem you have not properly diagnosed. Three major categories of data quality problems were found:

2.1.1 Inconsistent Regional Labels

The sales log recorded each transaction's region (where it was sold) as free text typed in by different sources. This meant the same region could appear under several different spellings and capitalisations — for example, "North", "NORTH", and even lowercase "north" all referred to the same place, but a computer treats each of these as a completely different value unless told otherwise. Left uncorrected, this would have silently split what should be one region's sales figures into three or four separate, smaller — and misleadingly weaker-looking — totals.

	A	B	C	D	E	F	G
11	ORD-01010	2026-03-03	PROD-012	8	South	110.2932534	
12	ORD-01011	06/14/2026	PROD-027	4	East	299.6538486	
13	ORD-01012	2026-02-20	PROD-019	7	North	101.2065899	
14	ORD-01013	2026-06-21	PROD-022	2	East	172.4667025	
15	ORD-01014	2026-06-01	PROD-024	7	NORTH	428.9647988	
16	ORD-01015	2026-02-28	PROD-025	5	East	111.4671465	
17	ORD-01016	2026-04-28	PROD-007	5	South	44.86755973	
18	ORD-01017	2026-06-09	PROD-020	6	South	43.81372038	
19	ORD-01018	2026-04-06	PROD-006	9	NORTH	470.984415	
20	ORD-01019	2026-06-29	PROD-021	6	WEST	258.3910146	
21	ORD-01020	2026-04-23	PROD-024	8	East	210.6119152	
22	ORD-01021	03/03/2026	PROD-001	7	WEST	407.3305047	
23	ORD-01022	2026-02-21	PROD-017	9	WEST	419.5547047	
24	ORD-01023	2026-01-12	PROD-029	4	south	172.7736062	
25	ORD-01024	2026-02-08	PROD-011	2	NORTH	349.8728482	
26	ORD-01025	2026-05-10	PROD-022	7	North	387.8501798	
27	ORD-01026	2026-05-11	PROD-026	6	South	330.7788366	
28	ORD-01027	2026-04-23	PROD-028	1	South	84.28158665	
29	ORD-01028	2026-04-11	PROD-018	9	NORTH	439.1827109	
30	ORD-01029	2026-04-23	PROD-021	9	North	274.1563308	
31	ORD-01030	2026-03-22	PROD-017	2	South	148.4114889	
32	ORD-01031	04/23/2026	PROD-007	3	NORTH	218.3618109	
33	ORD-01032	2026-01-02	PROD-017	5	South	28.4098343	
34	ORD-01033	2026-05-10	PROD-020	5	WEST	72.6547326	
35	ORD-01034	2026-02-23	PROD-009	1	East	385.1179078	
36	ORD-01035	2026-03-28	PROD-017	8	East	10.00570103	
37	ORD-01036	2026-05-09	PROD-020	8	South	214.117256	
38	ORD-01037	2026-05-27	PROD-020	4	South	266.8009987	

Figure 1 — Exploratory Structural Diagnostic Audit, showing the raw, unrefined Sales Log with inconsistent region spellings (e.g. "south" vs "North" vs "WEST") and mixed date formats.

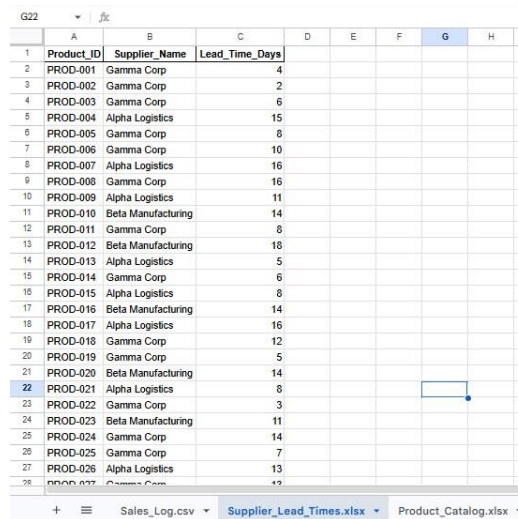
2.1.2 Mixed Date Formats

Order dates in the raw sales log were entered in two different formats within the very same column — some rows used the international standard (Year-Month-Day, e.g. 2026-03-03) while others used the American style (Month/Day/Year, e.g. 04/23/2026). A spreadsheet or database sorting these dates natively, without correction,

would sort them completely out of chronological order, making any time-based trend analysis meaningless or actively misleading.

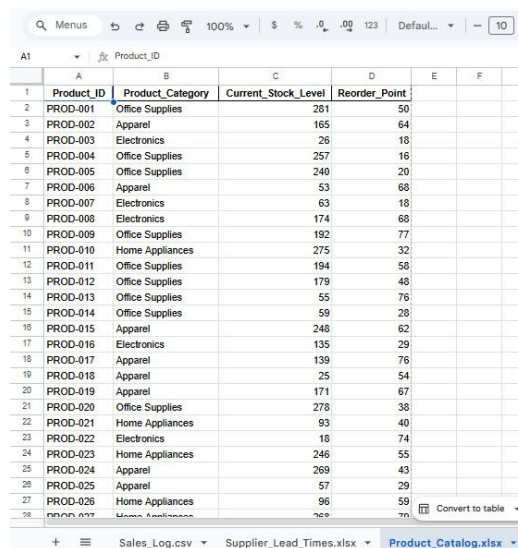
2.1.3 Missing Values (Data Gaps)

Some rows in the sales log were missing either the unit price or the quantity sold — for example, due to a manual entry being skipped. If these blank cells were left as they were, any total revenue calculation built on top of them would either error out or silently under-report the true sales figures, hiding some of the business's real performance.



	A	B	C	D	E	F	G	H
	Product_ID	Supplier_Name	Lead_Time_Days					
1	PROD-001	Gamma Corp	4					
2	PROD-002	Gamma Corp	2					
3	PROD-003	Gamma Corp	6					
4	PROD-004	Alpha Logistics	15					
5	PROD-005	Gamma Corp	8					
6	PROD-006	Gamma Corp	10					
7	PROD-007	Alpha Logistics	16					
8	PROD-008	Gamma Corp	16					
9	PROD-009	Alpha Logistics	11					
10	PROD-010	Beta Manufacturing	14					
11	PROD-011	Gamma Corp	8					
12	PROD-012	Beta Manufacturing	18					
13	PROD-013	Alpha Logistics	5					
14	PROD-014	Gamma Corp	6					
15	PROD-015	Alpha Logistics	8					
16	PROD-016	Beta Manufacturing	14					
17	PROD-017	Alpha Logistics	16					
18	PROD-018	Gamma Corp	12					
19	PROD-019	Gamma Corp	5					
20	PROD-020	Beta Manufacturing	14					
21	PROD-021	Alpha Logistics	8					
22	PROD-022	Gamma Corp	3					
23	PROD-023	Beta Manufacturing	11					
24	PROD-024	Gamma Corp	14					
25	PROD-025	Gamma Corp	7					
26	PROD-026	Alpha Logistics	13					
27	PROD-027	Gamma Corp	12					

Figure 2 — The Supplier Lead Time file, listing each product's assigned supplier and average delivery lead time in days — kept in a completely separate spreadsheet from the sales data.



	A	B	C	D	E	F	G
	Product_ID	Product_Category	Current_Stock_Level	Reorder_Point			
1	PROD-001	Office Supplies	281	50			
2	PROD-002	Apparel	165	64			
3	PROD-003	Electronics	26	18			
4	PROD-004	Office Supplies	257	16			
5	PROD-005	Office Supplies	240	20			
6	PROD-006	Apparel	53	68			
7	PROD-007	Electronics	63	18			
8	PROD-008	Electronics	174	68			
9	PROD-009	Office Supplies	192	77			
10	PROD-010	Home Appliances	275	32			
11	PROD-011	Office Supplies	194	58			
12	PROD-012	Office Supplies	179	48			
13	PROD-013	Office Supplies	55	76			
14	PROD-014	Office Supplies	59	28			
15	PROD-015	Apparel	248	62			
16	PROD-016	Electronics	135	29			
17	PROD-017	Apparel	139	76			
18	PROD-018	Apparel	25	54			
19	PROD-019	Apparel	171	67			
20	PROD-020	Office Supplies	278	38			
21	PROD-021	Home Appliances	93	40			
22	PROD-022	Electronics	18	74			
23	PROD-023	Home Appliances	246	55			
24	PROD-024	Apparel	269	43			
25	PROD-025	Apparel	57	29			
26	PROD-026	Home Appliances	96	59			
27	PROD-027	Home Appliances	100	70			

Figure 3 — The Product Catalog file, listing each product's category, current warehouse stock level, and reorder point — also kept separately from the sales log.

Because these three data sources — sales, product/warehouse data, and supplier data — lived in three separate files with no shared, cleaned link between them, nobody could easily answer a simple but important question:

"which specific products, from which specific supplier, are about to run out of stock, and how long will it take to get more?" Answering that question required building the pipeline described in the next section.

3. Solution Architecture & Technical Stack

To solve the problem above, the project was built using three layers, each handling a different job — much like a factory has a receiving dock, an assembly line, and a showroom.

Metric	What It Means, In Plain English
Data Engineering Layer	Python 3, using the Pandas and NumPy libraries, run inside Google Colab (an online Python notebook) and Anaconda Jupyter Notebook. This layer is the 'assembly line' — it reads the messy raw files, fixes their problems, and calculates business metrics.
Analytics Layer	Automated mathematical calculations and grouping logic applied directly within the Python pipeline to turn raw numbers into meaningful business measurements.
Presentation Layer	Google Sheets (with WPS Office also supported), used to build an interactive, multi-section dashboard that business users can filter and explore without needing to know how to code.

3.1 Building Realistic Test Data (Mock Data Simulation)

Before working with any real business data, the pipeline's reliability first had to be proven using safely generated, fake — but realistic — sample data. A dedicated script was written to programmatically create three files representing a fictional company's records: a product catalogue, a supplier lead-time list, and a sales log of 1,000 transactions. This matters because it let the pipeline be tested and proven under controlled conditions before being trusted with real figures.

```
[1]
✓ 0s
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# Ensure identical data generation every run
np.random.seed(42)
records_count = 1000

# 1. Create Product Catalog
products = [f"PROD-{i:03d}" for i in range(1, 31)]
categories = ['Electronics', 'Home Appliances', 'Office Supplies', 'Apparel']
catalog_data = {
    'Product_ID': products,
    'Product_Category': np.random.choice(categories, size=30),
    'Current_Stock_Level': np.random.randint(5, 300, size=30),
    'Reorder_Point': np.random.randint(15, 80, size=30)
}
pd.DataFrame(catalog_data).to_excel('Product_Catalog.xlsx', index=False)

# 2. Create Supplier Data
supplier_data = {
    'Product_ID': products,
    'Supplier_Name': np.random.choice(['Alpha Logistics', 'Beta Manufacturing', 'Gamma Corp'], size=30),
    'Lead_Time_Days': np.random.randint(2, 20, size=30)
}
pd.DataFrame(supplier_data).to_excel('Supplier_Lead_Times.xlsx', index=False)
```

Figure 4 — The Python script used to generate the Product Catalog and Supplier Lead Times mock data, with a fixed random seed so the same test data can be reproduced every time.

Crucially, the script did not just generate clean numbers — it deliberately injected realistic mistakes into the sales log on purpose, such as inconsistent region capitalisation, mixed date formats, and randomly blanked-out prices and quantities. This is a standard, responsible data engineering practice: it stress-tests the cleaning

pipeline against the exact kinds of errors it will eventually face in real life, before real business data is ever involved.

```
[1]
✓ Os
# 3. Create Messy Sales Log
order_dates = [datetime(2026, 1, 1) + timedelta(days=int(np.random.randint(0, 180))) for _ in range(records_count)]
# Mix up date formatting style to simulate manual entry bugs
formatted_dates = [d.strftime('%Y-%m-%d') if i % 10 != 0 else d.strftime('%m/%d/%Y') for i, d in enumerate(order_dates)]

sales_data = {
    'Order_ID': [f"ORD-{i:05d}" for i in range(1001, 1001 + records_count)],
    'Order_Date': formatted_dates,
    'Product_ID': np.random.choice(products, size=records_count),
    'Quantity_Sold': np.random.randint(1, 10, size=records_count),
    'Store_Region': np.random.choice(['NORTH', 'south', 'East', 'WEST', 'North', 'South'], size=records_count),
    'Unit_Price': np.random.uniform(10.0, 500.0, size=records_count)
}
df_sales = pd.DataFrame(sales_data)

# Inject empty fields (NaNs) to give yourself data cleaning work
df_sales.loc[df_sales.sample(frac=0.02).index, 'Unit_Price'] = np.nan
df_sales.loc[df_sales.sample(frac=0.01).index, 'Quantity_Sold'] = np.nan

df_sales.to_csv('Sales_Log.csv', index=False)
print("Success! 'Sales_Log.csv', 'Product_Catalog.xlsx', and 'Supplier_Lead_Times.xlsx' are ready.")

... Success! 'Sales_Log.csv', 'Product_Catalog.xlsx', and 'Supplier_Lead_Times.xlsx' are ready. ✓
```

Figure 5 — The section of the script that deliberately creates messy data: mixing date formats, randomising regional capitalisation, and injecting blank (missing) values into 1-2% of rows.

	A	B	C	D	E	F	G	H
1	Product_ID	Supplier_Name	Lead_Time_Days					
2	PROD-001	Gamma Corp	4					
3	PROD-002	Gamma Corp	2					
4	PROD-003	Gamma Corp	6					
5	PROD-004	Alpha Logistics	15					
6	PROD-005	Gamma Corp	8					
7	PROD-006	Gamma Corp	10					
8	PROD-007	Alpha Logistics	16					
9	PROD-008	Gamma Corp	16					
10	PROD-009	Alpha Logistics	11					
11	PROD-010	Beta Manufacturing	14					
12	PROD-011	Gamma Corp	8					
13	PROD-012	Beta Manufacturing	18					
14	PROD-013	Alpha Logistics	5					
15	PROD-014	Gamma Corp	6					
16	PROD-015	Alpha Logistics	8					
17	PROD-016	Beta Manufacturing	14					
18	PROD-017	Alpha Logistics	16					
19	PROD-018	Gamma Corp	12					
20	PROD-019	Gamma Corp	5					
21	PROD-020	Beta Manufacturing	14					
22	PROD-021	Alpha Logistics	8					
23	PROD-022	Gamma Corp	3					
24	PROD-023	Beta Manufacturing	11					
25	PROD-024	Gamma Corp	14					
26	PROD-025	Gamma Corp	7					
27	PROD-026	Alpha Logistics	13					
28	PROD-027	Gamma Corp	12					

Figure 6 — The resulting raw Sales Log file, viewed in Google Sheets, showing the intentionally inconsistent region text ("North", "NORTH", "south", "WEST") and mixed date formats ("2026-03-03" next to "03/03/2026") that the cleaning pipeline had to correct.

4. The Algorithmic Data Engineering Pipeline

This is the heart of the project — the automated engine that turns the three messy, separate files into one single, clean, reliable dataset. It is written entirely in Python and runs from start to finish with a single command, so the same cleaning process can be repeated reliably every time new data arrives, without any manual spreadsheet editing.

4.1 Step-by-Step Breakdown of the Cleaning Process

1. Load the files: The pipeline reads all three raw files directly — the Sales Log (a CSV file), the Product Catalog (an Excel file), and the Supplier Lead Times (another Excel file) — into the Python environment, ready for processing.
2. Standardise text formatting: Every text value that will later be used to match or group records — such as region names and product ID codes — is automatically converted to uppercase and has any accidental extra spaces removed. This single step is what fixes the "North" vs "NORTH" vs "south" problem described earlier, because after this step, all three become simply "NORTH", allowing them to be correctly grouped together as one region.
3. Fix the mixed date formats: The order date column is passed through a flexible date-parsing function that can recognise multiple date formats at once and convert every single one into one uniform, computer-standard date format. This resolves the earlier sorting and trend-analysis problem entirely.
4. Fill in missing values responsibly (imputation): Rather than deleting rows with missing prices or quantities — which would throw away potentially valuable sales records — the pipeline fills the gaps intelligently. Missing quantities are filled in using the middle (median) value from the rest of that column, and missing prices are filled in using the average price previously recorded specifically for that same product. This approach preserves every row of real data while avoiding wildly inaccurate guesses.
5. Calculate total transaction revenue: For every single sales row, the pipeline multiplies the quantity sold by the unit price to produce a Line Revenue figure — the actual money that transaction brought in.
6. Merge all three files into one master table: Using the shared Product ID present in all three files as the connecting key, the pipeline joins the sales log, the product catalogue, and the supplier lead times into a single unified table, so that every sales transaction now also shows its product's current stock level, its reorder point, its supplier's name, and that supplier's average delivery time — all in one place.

```

import pandas as pd
import numpy as np

print("Starting the Data Cleaning & Metrics Engine...")

# 1. Load the files directly from your Colab workspace
df_sales = pd.read_csv('Sales_Log.csv')
df_product = pd.read_excel('Product_Catalog.xlsx')
df_supplier = pd.read_excel('Supplier_Lead_Times.xlsx')

# 2. Text Standardization (Force everything to clean, matching UPPERCASE)
df_sales['Store_Region'] = df_sales['Store_Region'].astype(str).str.upper().str.strip()
df_sales['Product_ID'] = df_sales['Product_ID'].astype(str).str.upper().str.strip()
df_product['Product_ID'] = df_product['Product_ID'].astype(str).str.upper().str.strip()
df_supplier['Product_ID'] = df_supplier['Product_ID'].astype(str).str.upper().str.strip()

# 3. Standardize the broken date column
df_sales['Order_Date'] = pd.to_datetime(df_sales['Order_Date'], errors='coerce', format='mixed')

# 4. Handle Missing Values (Imputation)
# Fill empty sales quantities with the median value of that column
df_sales['Quantity_Sold'] = df_sales['Quantity_Sold'].fillna(df_sales['Quantity_Sold'].median())

# Fill missing prices by finding the calculated average price for that specific product ID
mean_prices = df_sales.groupby('Product_ID')['Unit_Price'].transform('mean')
df_sales['Unit_Price'] = df_sales['Unit_Price'].fillna(mean_prices)

# Calculate total transaction revenue line items
df_sales['Line_Revenue'] = df_sales['Quantity_Sold'] * df_sales['Unit_Price']

# 5. Combine all 3 tables cleanly using Inner Joins on 'Product_ID'
df_master = pd.merge(df_sales, df_product, on='Product_ID', how='inner')
df_master = pd.merge(df_master, df_supplier, on='Product_ID', how='inner')

```

Figure 7 — The core cleaning script: loading the three files, standardising text case, fixing mixed date formats, filling in missing prices and quantities, and merging everything into one master table using Product ID.

4.2 Business Metrics Calculated Automatically

Once the data was clean and merged, the pipeline calculated three additional, business-critical measurements automatically for every single product. These numbers are what actually make the dashboard useful — they turn raw transaction data into clear, actionable business intelligence.

4.2.1 Inventory Turnover Rate (ITR)

This measures how efficiently a product is selling and being restocked, relative to how much of it is currently sitting in the warehouse. In plain terms: a high turnover rate means a product is selling fast and stock keeps moving; a low rate means stock is piling up and money is sitting idle on the shelf. It is calculated as the total units sold in a year divided by the current stock level.

4.2.2 Stockout Risk Alert Flag

This is a simple yes/no warning flag automatically raised for any product whose current stock level has fallen below its predefined "reorder point" — the minimum safe stock level set for that item. If the flag is raised (shown as the number 1), it means the product needs to be reordered urgently before it runs out completely and disappoints a customer.

4.2.3 Programmatic Pareto Distribution (ABC Analysis)

This groups every product into one of three priority tiers based on how much revenue it brings in, using a well-known business principle sometimes called the "80/20 rule". In simple terms:

- Class A — the small group of products responsible for roughly the top 80% of total revenue. These are the business's core money-makers and deserve the strictest stock protection.
- Class B — the next group of products responsible for the following 15% of revenue. These are stable, steady sellers needing standard, routine review.
- Class C — the remaining products responsible for only the last 5% of revenue. These are low-priority, slow-moving items that may be candidates for reducing stock or discontinuing altogether.

```

# 6. Calculate Inventory Turnover Rate (Total annual units sold / current stock levels)
total_sold = df_master.groupby('Product_ID')['quantity_sold'].transform('sum')
df_master['Inventory_Turnover_Rate'] = round(total_sold / df_master['Current_Stock_Level'], 2)

# 7. Create a Stockout Risk Alert Flag (1 if current stock is lower than our safety reorder boundary)
df_master['Stockout_Risk_Flag'] = np.where(df_master['Current_Stock_Level'] < df_master['Reorder_Point'], 1, 0)

# 8. Run an ABC Analysis (Categorizing our top revenue drivers via a 20% / 15% / 5% Pareto distribution)
prod_rev = df_master.groupby('Product_ID')['Line_Revenue'].sum().reset_index()
prod_rev = prod_rev.sort_values(by='Line_Revenue', ascending=False)
prod_rev['Cumulative_Rev'] = prod_rev['Line_Revenue'].cumsum()
total_pool = prod_rev['Line_Revenue'].sum()
prod_rev['Revenue_Pct'] = prod_rev['Cumulative_Rev'] / total_pool

def assign_abc_class(percentage):
    if percentage <= 0.80: return 'A' # Generates top 80% of money
    elif percentage <= 0.95: return 'B' # Generates middle 15%
    else: return 'C' # Generates bottom 5% tail end

prod_rev['ABC_Classification'] = prod_rev['Revenue_Pct'].apply(assign_abc_class)

# Map classifications back into our main operational file
abc_map = prod_rev.set_index('Product_ID')['ABC_Classification'].to_dict()
df_master['ABC_Classification'] = df_master['Product_ID'].map(abc_map)

# 9. Save your structured portfolio master dataset
df_master.to_csv('Clean_Sales_Inventory.csv', index=False)
print("Pipeline Success! 'Clean_Sales_Inventory.csv' is generated and saved in your files panel.")

... Starting the Data Cleaning & Metrics Engine...
Pipeline Success! 'Clean_Sales_Inventory.csv' is generated and saved in your files panel.

```

Figure 8 — The Python code calculating Inventory Turnover Rate, the Stockout Risk Alert Flag, and the ABC Classification (Pareto analysis), then saving the final, clean master dataset as Clean_Sales_Inventory.csv.

At the very end of this stage, the pipeline exports one single, verified, ready-to-use file — Clean_Sales_Inventory.csv — which becomes the trusted single source of truth for everything shown on the dashboard in the next section.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
	Order_ID	Order_Date	Product_ID	Quantity_Sold	Stock_Level	Unit_Price	Line_Revenue	Product_Category	Current_Stock_Level	Reorder_Point	Supplier_Name	Lead_Time_Days	Inventory_Turnover_Rate	Stockout_Risk_Flag	ABC_Classification	
1	ORD-01001	2026-05-09	PROD-002	3	NORTH	148.4230723	445.269217	Apparel	165	64	Gamma Corp	2	1.32	0	A	
2	ORD-01002	2026-05-01	PROD-007	3	NORTH	162.9946919	548.9640757	Electronics	63	18	Alpha Logistics	16	2.59	0	B	
3	ORD-01003	2026-01-27	PROD-016	6	WEST	448.1062062	2088.637249	Apparel	25	54	Gamma Corp	12	7.64	1	A	
4	ORD-01004	2026-05-01	PROD-027	3	NORTH	473.7638914	1421.291074	Home Appliances	268	79	Gamma Corp	13	0.69	0	A	
5	ORD-01005	2026-04-26	PROD-002	2	SOUTH	447.353316	684.707628	Apparel	165	64	Gamma Corp	2	1.32	0	A	
6	ORD-01006	2026-01-03	PROD-027	5	NORTH	215.529517	1077.647585	Home Appliances	268	79	Gamma Corp	13	0.69	0	A	
7	ORD-01007	2026-04-13	PROD-017	5	NORTH	392.3790965	1981.895482	Apparel	139	76	Alpha Logistics	16	1.62	0	A	
8	ORD-01008	2026-06-04	PROD-011	1	WEST	243.4045093	243.4045093	Office Supplies	194	58	Gamma Corp	8	0.71	0	A	
9	ORD-01009	2026-05-17	PROD-027	3	NORTH	253.7945276	761.3635027	Home Appliances	268	79	Gamma Corp	13	0.69	0	A	
10	ORD-01010	2026-03-03	PROD-012	8	SOUTH	110.2932534	882.3460275	Office Supplies	179	48	Beta Manufacturing	18	1.24	0	A	
11	ORD-01011	2026-06-14	PROD-027	4	EAST	299.6533466	1198.615394	Home Appliances	268	79	Gamma Corp	13	0.69	0	A	
12	ORD-01012	2026-02-20	PROD-019	7	NORTH	101.2065899	708.4481283	Apparel	171	67	Gamma Corp	5	0.97	0	B	
13	ORD-01013	2026-06-21	PROD-022	2	EAST	172.4667025	344.9334051	Electronics	18	74	Gamma Corp	3	9.67	1	B	
14	ORD-01014	2026-06-01	PROD-024	7	NORTH	428.9647868	3002.753592	Apparel	269	43	Gamma Corp	14	0.69	0	A	
15	ORD-01015	2026-02-20	PROD-025	5	EAST	111.4071465	557.3557326	Apparel	57	29	Gamma Corp	7	2.98	0	A	
16	ORD-01016	2026-04-25	PROD-007	5	SOUTH	44.8075973	224.3377887	Electronics	63	18	Alpha Logistics	16	2.59	0	B	
17	ORD-01017	2026-06-09	PROD-030	6	SOUTH	43.81372038	262.8823223	Office Supplies	278	38	Beta Manufacturing	14	0.53	0	A	
18	ORD-01018	2026-04-06	PROD-006	9	NORTH	470.984415	4238.859735	Apparel	53	68	Gamma Corp	10	2.85	1	A	
19	ORD-01019	2026-06-29	PROD-021	6	WEST	258.3910166	1550.346088	Home Appliances	93	40	Alpha Logistics	8	1.61	0	A	
20	ORD-01020	2026-04-23	PROD-024	9	EAST	210.6191952	1684.995322	Apparel	269	43	Gamma Corp	14	0.69	0	A	
21	ORD-01021	2026-03-03	PROD-001	7	WEST	407.3305947	2851.313533	Office Supplies	281	50	Gamma Corp	4	0.47	0	A	
22	ORD-01022	2026-02-21	PROD-017	9	WEST	419.5547047	3775.962342	Apparel	139	76	Alpha Logistics	16	1.62	0	A	
23	ORD-01023	2026-01-12	PROD-029	4	SOUTH	172.7736962	891.094249	Apparel	210	15	Gamma Corp	8	0.94	0	A	
24	ORD-01024	2026-02-08	PROD-011	2	NORTH	349.8724462	699.1659983	Office Supplies	194	58	Gamma Corp	8	0.71	0	A	
25	ORD-01025	2026-05-10	PROD-022	7	NORTH	387.8501798	2714.951258	Electronics	18	74	Gamma Corp	3	9.67	1	B	
26	ORD-01026	2026-05-11	PROD-026	6	SOUTH	330.7783666	1984.873019	Home Appliances	96	59	Alpha Logistics	13	1.8	0	A	
27	ORD-01027	2026-04-10	PROD-018	4	EAST	243.7845666	814.7845666	Home Appliances	93	40	Alpha Logistics	8	1.61	0	A	

Figure 9 — The final, cleaned master dataset (15,015 rows), now containing every original sales record enriched with Current Stock Level, Reorder Point, Supplier Name, Lead Time, Inventory Turnover Rate, Stockout Risk Flag, and ABC Classification — all in one table.

5. Dashboard Architecture & Layout Blueprint

With the clean master data ready, the final stage was to present it in a way that a busy manager could understand at a glance, without needing to read spreadsheets or write any code. The dashboard was built in Google Sheets, arranged in three clear rows, plus one interactive control that ties the whole page together.

5.1 Row 1 — Multi-Tier Strategic KPI Banner

At the very top of the dashboard sit three large, easy-to-read scorecards summarising the whole business at a glance: total revenue earned, total units sold, and — most importantly — a live count of high-priority products currently at risk of running out of stock. This last figure is deliberately highlighted in a soft red colour whenever the number is greater than zero, so it immediately catches a manager's eye without them needing to dig for it.

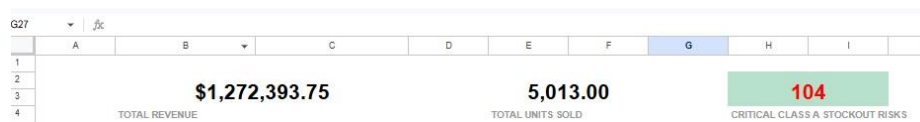


Figure 10 — The top KPI banner: Total Revenue (\$1,272,393.75), Total Units Sold (5,013), and 104 Critical Class A Stockout Risks, flagged in red for immediate attention.

In plain terms, this banner answers three questions in under two seconds: "How much money have we made? How much have we sold? And is anything urgent happening right now?"

5.2 Row 2 — Tactical Supply vs. Demand Velocity Trends

Below the scorecards sits a combination chart comparing two things over time on the same timeline: how much revenue is being earned (shown as blue bars) against how much stock is sitting in the warehouse (shown as a red line). Because these two measurements use very different scales — money in thousands of dollars versus item counts — the chart uses two separate vertical axes (a dual-axis chart), so both lines stay readable and comparable side by side.

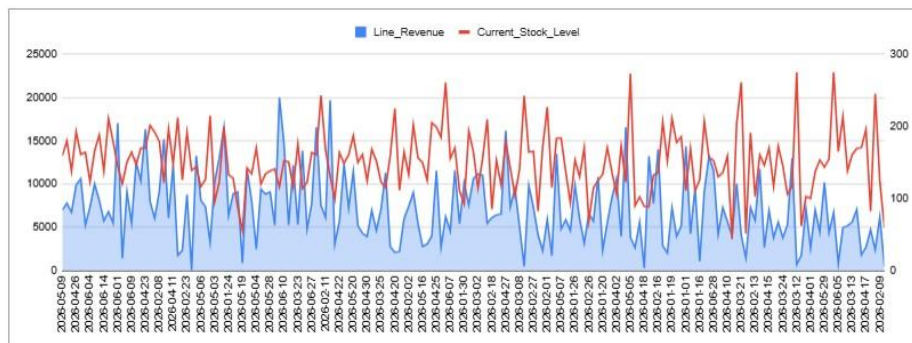


Figure 11 — Dual-axis combo chart: blue columns show daily revenue (Line_Revenue) against the left axis; the red line shows average current stock level (Current_Stock_Level) against the right axis, revealing seasonal demand patterns over time.

This chart helps a manager visually spot patterns — for example, moments where sales are spiking (tall blue bars) at the same time stock is dropping sharply (a falling red line), which is an early visual warning sign of an approaching stockout well before it would show up as a hard number.

5.3 Row 3 — Actionable Operational Intelligence Tables

The dashboard's third section is a focused table — technically built using a Google Sheets Pivot Table — that automatically filters out every supplier and product that is currently safe, showing only the suppliers connected to active stock shortfalls, sorted by which supplier takes the longest to deliver.

Supplier_Name	AVERAGE of Lead_Time_Days	SUM of Quantity
Gamma Corp	8.378640777	516
Alpha Logistics	5	162
Grand Total	7.503597122	678

Figure 12 — The Fulfillment Exception Pivot Table, showing only suppliers tied to current stock shortfalls: Gamma Corp (8.38-day average lead time, 516 affected units) and Alpha Logistics (5-day average lead time, 162 affected units).

This table immediately identifies Gamma Corp as the single biggest operational risk in the whole supply chain: it is both the slowest supplier (averaging 8.38 days to deliver) and the one tied to by far the greatest number of at-risk product units (516, compared to only 162 for Alpha Logistics). In plain terms, this one table alone tells a manager exactly which vendor conversation needs to happen first.

5.4 Interactive Filtering — The Store Region Slicer

Finally, a single filtering control called a "slicer" is pinned prominently on the dashboard, allowing anyone viewing it to instantly filter every chart, scorecard, and table on the page down to just one regional store — without touching any formulas or reloading any data.

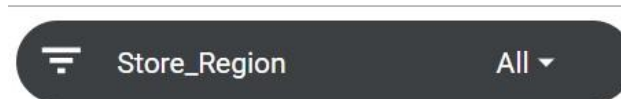


Figure 13 — The Store Region slicer control. Selecting a specific region here instantly updates the KPI banner, the trend chart, and the supplier risk table together, all at once.

This means a regional manager responsible for, say, only the WEST region, can click that single filter and immediately see a version of the entire dashboard scoped only to their own area of responsibility, with no extra work required.

6. How Everything Connects — A Simple Walkthrough

For a reader unfamiliar with data projects, it can help to see the whole journey laid out as one simple, ordered story, from raw data to final decision:

1. Three separate, messy spreadsheets exist: a sales log, a product/warehouse file, and a supplier delivery-time file.
2. A Python script automatically reads all three files at once.
3. The script fixes spelling and capitalisation inconsistencies, fixes mixed date formats, and fills in any missing prices or quantities using sensible averages — without deleting any real data.
4. The script joins all three files together into one single, unified table, using each product's unique ID code as the connecting link.
5. The script calculates three new business metrics for every product: how fast it turns over, whether it is at risk of running out, and how valuable it is to the business overall (its ABC class).
6. The finished, clean table is saved as one master file, `Clean_Sales_Inventory.csv`.
7. That master file feeds a live, interactive dashboard in Google Sheets, built from three visual sections plus one region filter.
8. A manager opens the dashboard, sees at a glance that 104 top-priority products are at risk, filters to their supplier report, and immediately identifies Gamma Corp as the supplier responsible for the majority of that risk.
9. Armed with that clear, specific insight, the business can now act — for example, by renegotiating delivery terms with that one supplier — instead of guessing or discovering the problem only after a stockout has already happened.

7. Conclusion

The Supply Chain & Sales Intelligence System demonstrates, from start to finish, the real business value of turning fragmented, error-prone raw data into a single, trustworthy analytics system. By building a robust, repeatable data pipeline, this project eliminated manual data-entry inconsistencies, standardised regional and date formatting across every record, and unified three previously disconnected operational files into one clean, reliable master data layer.

Beyond the technical cleanup, the project's real achievement is turning that clean data into decisions a business can actually act on: identifying exactly which 104 products are at immediate risk, and pinpointing exactly which supplier — Gamma Corp — is the primary cause of that risk, all visible on a single, easy-to-read dashboard screen.

8. Strategic Recommendations

Based on the insights the dashboard surfaced, the following concrete actions are recommended to business leadership:

8.1 Vendor Service Level Agreement (SLA) Renegotiation

Procurement teams should prioritise renegotiating delivery turnaround times and contract terms with Gamma Corp specifically. Their documented average delivery delay of 8.38 days is directly linked to 516 high-priority stockout risks, making them the single largest source of supply chain friction identified in this analysis.

8.2 Capital Reallocation via ABC Segmentation

Operations teams should reduce safety stock holding levels for Class C (long-tail, low-revenue) items. Reducing or liquidating slow-moving inventory frees up working capital — money that can then be redirected to better support the fast-moving, high-margin Class A products that generate the bulk of the business's revenue.

8.3 Safety Stock Buffer Adjustments

For Class A products that show strong seasonal demand swings on the trend chart, reorder point thresholds should be temporarily increased by 15% during peak sales months. This builds in a protective buffer and helps prevent the kind of costly stockout-driven revenue loss this project was specifically designed to catch early.

Appendix A: Technical Stack Summary

Layer	Tools Used	Purpose
Data Engineering	Python 3, Pandas, NumPy, OpenPyXL	Reading, cleaning, merging, and calculating metrics from raw files
Development Environment	Google Colab, Anaconda Jupyter Notebook	Where the Python pipeline was written and executed

Layer	Tools Used	Purpose
Analytics	Automated grouping & mathematical transformations	Turning cleaned raw data into business metrics (ITR, Stockout Flag, ABC Class)
Presentation	Google Sheets / WPS Office	Interactive, filterable executive dashboard for non-technical users

Appendix B: Key Output Files

File Name	Description
Sales_Log.csv	Raw, unrefined transaction-level sales records (before cleaning)
Product_Catalog.xlsx	Product categories, current stock levels, and reorder points
Supplier_Lead_Times.xlsx	Supplier assigned to each product and average delivery lead time
Clean_Sales_Inventory.csv	Final, cleaned, merged master dataset with all calculated metrics — the single source of truth powering the dashboard
Executive_Dashboard (Google Sheet tab)	The finished, interactive business intelligence dashboard

Appendix C: Full Step-by-Step Replication Manual

This appendix reproduces the full chapter-by-chapter replication manual for the project, in its original structure and phase order, for anyone who wants to rebuild the system from scratch.

Chapter 1: Executive Summary & Strategic Logistical Mission

1.1 Commercial Introduction

In modern enterprise environments, supply chains generate vast pools of operational records, logistics turnaround times, and warehouse stock logs. However, raw data is rarely born clean. It is typically collected across disconnected spreadsheet sheets, mixed file formats, and systems prone to human manual input errors.

The Supply Chain & Sales Intelligence System is an end-to-end data transformation pipeline and executive reporting workspace designed to bridge these corporate informational silos. This project establishes an automated pipeline to handle missing values, correct text casing inconsistencies, and format disparate tables into a unified data storage layer. The resulting master dataset drives a responsive reporting workspace that converts row-level operations data into a strategic business asset.

1.2 Core Performance Metrics Framework

To provide management with actionable insights rather than a simple log of historical entries, the system calculates advanced operational supply chain equations directly across combined master fields:

A. Inventory Turnover Rate (ITR): Measures capital liquidity and asset velocity by quantifying how efficiently stock is sold and replenished relative to current holding capacity.

Formula: $\text{Annualized Units Sold} / \text{Current Stock Level}$

B. Stock-out Risk Alert Flag: A programmable binary threshold rule that isolates fulfillment shortfalls by flagging items the absolute moment their warehouse count drops lower than local safety stock parameters.

Formula: $1 \text{ if Current Stock Level} < \text{Reorder Point, else } 0$

C. Algorithmic Pareto ABC Segmentation: Applies the 80/20 Pareto Principle to group stock profiles by economic value based on cumulative financial contribution. This helps procurement teams prioritize high-value stock lines:

- Class A (Top 80%): Core strategic revenue drivers requiring strict protection.
- Class B (Next 15%): Stable operational items managed under standard schedules.
- Class C (Bottom 5%): Long-tail low-priority items suitable for liquidation.

Chapter 2: Solution Architecture & End-to-End Flow Data Map

2.1 Ingestion Layer

Ingests unrefined source sheets containing structural errors from loose files (Sales_Log.csv, Product_Catalog.xlsx, and Supplier_Lead_Times.xlsx) into memory using Python Pandas.

2.2 Core Python Transformation Engine

Processes characters, dates, and empty rows. Text parameters are stripped of whitespaces and standardized to uppercase to eliminate merge fragmentation. Chronological vectors with mixed formatting delimiters are parsed into standardized datetime objects.

2.3 Downstream Reporting Presentation Layer

Exports a unified file (Clean_Sales_Inventory.csv) to populate a multi-tier, cross-filtered executive dashboard workspace equipped with conditional risk alert layers, trend visualizations, and supplier bottleneck pivot matrices.

Chapter 3: Stage 1 — Industrial Mock Data Simulation Logic

The data generation engine utilizes NumPy array distributions and Pandas libraries to engineer the raw testing tables. The generated assets include structural text variances, empty float markers, and randomized time string layouts to test the pipeline error-handling capabilities.

Chapter 4: Stage 2 — Excel Core Structural Diagnostics & Data Audit

4.1 Inconsistent Casing Discrepancies

Filtering the Store_Region column reveals text entry flaws where split text casings occur. A computer reads these as separate categorical entities, which fragments regional summary metrics.

4.2 Empty Fields & Missing Values

Sorting the transaction columns isolates null entries within Unit_Price and Quantity_Sold. Leaving these blank would break downstream aggregate operations and distort total revenue calculations.

4.3 Delimiter Formats and Temporal Chaos

Inspecting the Order_Date field exposes changing date string formats, alternating randomly between standard dash delimiters (2026-03-14) and localized slashes (04/18/2026). This prevents accurate historical sorting.

Chapter 5: Stage 3 — The Python Ingestion & Data Cleaning Engine

The automation backend carries out five distinct data processing phases:

1. Character uniform string normalization via (.astype(str).str.upper().str.strip()) syntax rows.
2. Chronological normalization using (pd.to_datetime) mixed matching criteria tools.
3. Median value imputation for missing sales quantities.
4. Group-by mean price imputation for empty financial columns.
5. Direct database relational table merges linked via the core Product_ID code block.

Chapter 6: Stage 4 — Front-End Executive Working Interface Blueprint

6.1 Row 1: Strategic Executive Scorecards

- Total Revenue Box (B2:C3): Merged card tracking metrics via formula =SUM(Clean_Master_Data!G:G).
- Total Volume Box (E2:F3): Merged cell counting throughput metrics via formula =SUM(Clean_Master_Data!D:D).
- High-Priority Safety Alerts Box (H2:I3): Counts Class A stocklines currently falling below boundaries: =COUNTIFS(Clean_Master_Data!L:L, 1, Clean_Master_Data!M:M, "A").

6.2 Row 2: Tactical Combo Trend Visualization Chart

- X-Axis: Order_Date vector grouped under checked data aggregation options.
- Primary Axis: Maps historical financial total performance values (Line_Revenue).
- Secondary Axis: Maps inventory capacity holding values (Current_Stock_Level) on the right vertical line scale.

6.3 Row 3: Operational Bottleneck Pivot Grid

- Rows Configuration: Set to map Supplier_Name data dimensions.
- Calculated Values Mapping: Configured to map Lead_Time_Days (Average) and Quantity_Sold (Sum).
- Filtering Matrix: Set to evaluate Stockout_Risk_Flag variables to filter out safe fields.

Chapter 7: Detailed Comprehensive Replication Step-by-Step Manual

7.1 Workspace Path Architecture

Create a folder on your computer disk space named exactly: Supply_Chain_Portfolio_System. Inside this root folder, create a nested folder structure: datasets/, output/, scripts/, and images/.

7.2 Running Ingestion Operations

Open your notebook environment, run your simulation scripts, and verify files appear in the data directory panel.

7.3 Dashboard Tab Configurations

Create a blank spreadsheet workbook container inside Google Sheets. Use the import tool to load your final data into a worksheet layer titled exactly: Clean_Master_Data.

Chapter 8: Project System Integration Conclusion

The implementation of this project demonstrates the clear business value of transforming siloed operational data into a unified analytics asset. By establishing a robust data engineering pipeline, this project successfully eliminated data entry defects, resolved missing fields, and unified logs into a single master data layer.

Chapter 9: Strategic Reconciliation & Executive Enterprise Recommendations

- **Vendor Service Level Agreement (SLA) Renegotiation:** Procurement teams must prioritize re-negotiating contract terms with Gamma Corp. Their documented average delivery delay of 8.38 days accounts for 516 high-priority stockout risks.
- **Capital Reallocation via ABC Segmentation:** Reduce safety stock holding thresholds for Class C long-tail items. Liquidating slow-moving inventory optimizes working capital, freeing up cash flow to back Class A lines.
- **Safety Stock Buffer Adjustments:** For Class A items with high seasonal demand variations, reorder points should be dynamically scaled up by 15% during peak sales months to build a buffer and prevent revenue leakage.